

Making a calendar, line by line

Everything that happens between clicking **Make it** and the calendar existing.

Thundermail calendars add-on, commit `ba21bf4` of `thunderbird/desktop-support-tools`. Two files: `firefox-addon/popup.js`, which is the add-on, and `caldav_account.js`, which is shared with the command-line tools and tested against the same fixtures as its Python twin. Every line number below is from that commit. Written 1 September 2026.

Fourteen steps, one request. Your password is read at step 5 and turned into a header at step 10. The calendar's name is read at step 4, becomes the address at step 7 and the request body at step 8. One `MKCALENDAR` goes out at step 12. Nothing else is sent, and nothing is kept afterwards.

STEP	WHERE
1 — the click	<code>popup.js:79–82</code>
2 — ask permission, before anything else	<code>popup.js:94–102</code>
3 — buttons off, errors caught	<code>popup.js:138–149</code>
4 — read the name	<code>popup.js:189–193</code>
5 — read the password	<code>popup.js:227–237</code>
6 — refuse to overwrite	<code>popup.js:195–209</code>
7 — the name becomes the address	<code>popup.js:198, 211</code> · <code>caldav_account.js:170–173</code>
8 — the name becomes the body	<code>caldav_account.js:55–61, 246–253</code>
9 — refuse another origin	<code>popup.js:256–266</code>
10 — the password becomes a header	<code>popup.js:268</code> · <code>caldav_account.js:239–244</code>
11 — a deadline	<code>popup.js:272–283</code>
12 — the request	<code>popup.js:285–304</code>
13 — the answer	<code>popup.js:306–318</code>
14 — what you see	<code>popup.js:215–223</code>

1 The click

`firefox-addon/popup.js:79–82`

```
79 makeForm.addEventListener("submit", (event) => {
80   event.preventDefault();
81   run(make, askedFor());
82 });
```

`preventDefault()` stops the browser navigating away, which is what a form normally does. Then two things happen in one line, and their order is the whole point: `askedFor()` runs *first*, and its result — a promise — is handed to `run()` along with the work to do.

2 Ask permission, before anything else

firefox-addon/popup.js:94–102

```
94 function askedFor() {
95   try {
96     permittedOrigin = new URL(document.querySelector("#home").value.trim()).origin;
97     return api.permissions.request({ origins: [`${permittedOrigin}/*`] });
98   } catch {
99     permittedOrigin = null;
100    // Not an address at all. Let credentials() say so in words, rather than
101    // failing here as a permission problem, which it is not.
102    return Promise.resolve(true);
```

Firefox grants a permission only while a user gesture is in hand, and the first `await` anywhere spends that gesture. So this must be the first thing the click does — before reading fields, before validating anything. It asks outright rather than checking first, because `permissions.contains()` is itself asynchronous and would cost us the right to ask; requesting one already granted resolves `true` and shows nobody anything.

Line 96 also records **which** origin was asked about. Step 9 depends on that.

3 Buttons off, errors caught

firefox-addon/popup.js:138–149

```
139   permitted = permission;
140   const buttons = [...document.querySelectorAll("button")];
141   buttons.forEach((button) => (button.disabled = true));
142   try {
143     await action();
144   } catch (error) {
145     say(error.message, "bad");
146   } finally {
147     buttons.forEach((button) => (button.disabled = false));
148   }
149 }
```

Every button is disabled for the duration, so a second click cannot start a second request on top of the first. Everything the rest of this walkthrough throws lands at line 145 and becomes a sentence in the status line — which is why the errors below are written as sentences rather than as codes.

4 Read the name

firefox-addon/popup.js:189–193

```

189 const name = document.querySelector("#name").value.trim();
190 if (!name) throw new Error("The calendar needs a name.");
191 if (home !== known.home || known.calendars.length === 0) {
192     throw new Error("List the calendars again first, so this can see what is already there.");
193 }

```

The name is read straight from the field, at this moment, and lives in a local `const`. Line 191 is a precondition rather than a formality: step 6 checks the new name against the calendars the account already has, and it can only do that if the listing on screen belongs to the same server that is about to be written to.

5 Read the password

firefox-addon/popup.js:227–237

```

227 function credentials() {
228     const home = document.querySelector("#home").value.trim();
229     const user = document.querySelector("#user").value.trim();
230     const password = document.querySelector("#password").value;
231     const problem = targetProblem(home);
232     if (problem) throw new Error(problem);
233     if (!user) throw new Error("Which username should this sign in with?");
234     if (!password) throw new Error("This needs your app password to ask the server anything.");
235     // A collection's address ends in a slash, and servers differ on whether they
236     // forgive a missing one. None mind an extra.
237     return { home: home.endsWith("/") ? home : `${home}/`, user, password };

```

This is the only place the password is read, and this is the only kind of moment it happens in — a click, on the way to a request. It comes out of the DOM into a local `const` and is returned to the caller. It is not copied into any variable that outlives the call, not written to storage, and not logged.

`targetProblem()` at line 231 refuses plain `http://` unless the server is on your own machine or network, so the password cannot be put on the wire unencrypted by mistake.

6 Refuse to overwrite

firefox-addon/popup.js:195–209

```

195 // Nothing is overwritten, and the check is the listing rather than the
196 // server's refusal: two calendars with one name are indistinguishable in
197 // Thunderbird's list, and the server has no opinion about that.
198 const segment = addressFor(name);
199 for (const calendar of known.calendars) {
200     if (calendar.path.split("/").pop().toLowerCase() === segment.toLowerCase()) {
201         throw new Error(`${calendar.path}/ is already there, called "${calendar.name}".`);
202     }
203     if (calendar.name.toLowerCase() === name.toLowerCase()) {
204         throw new Error(
205             `This account already has a calendar called "${calendar.name}". Two calendars with `+
206             "one name are indistinguishable in Thunderbird's list, so pick another.",
207         );
208     }
209 }

```

Two refusals, for two different reasons. Line 200 catches an address already in use — the server would refuse that too, with a 405, so this is only a nicer way to hear it. Line 203 catches a *name* already in use, which the server has no opinion about at all: it will happily hold two calendars called the same thing, and you would then be unable to tell them apart in Thunderbird's list.

7 The name becomes the address

caldav_account.js:170–173 and popup.js:211

```
170 export function addressFor(name) {
171   const slug = (name || "").replace(/^[A-Za-z0-9]+/g, "-").replace(/^-+|-+$/g, "").toLowerCase();
172   return slug || "calendar";
173 }

211   const address = new URL(`${segment}/`, home).href;
```

Everything that is not a letter or a digit becomes a hyphen, so `ticket 7067` becomes `ticket-7067` and `Réunion budgétaire!` becomes `r-union-budg-taire`. That is character for character what `address_for()` does in `caldav_make_calendar.py`, and a test asserts the two agree — otherwise a calendar made here and one made from the command line would land at different addresses.

It also means **the address cannot escape its place**: no slash, no colon and no dot can survive the substitution, so the name you type cannot reach outside the calendar home at line 211, whatever it says.

8 The name becomes the body

caldav_account.js:55–61 and 246–253

```
55 export function makeBody(name) {
56   return `<?xml version="1.0" encoding="utf-8"?>
57   <C:mkcalendar xmlns:D="DAV:" xmlns:C="urn:ietf:params:xml:ns:caldav">
58     <D:set><D:prop><D:displayname>${escapeXml(name)}</D:displayname></D:prop></D:set>
59   </C:mkcalendar>
60 `;
61 }

246 export function escapeXml(text) {
247   return String(text)
248     .replace(/&/g, "&amp;")
249     .replace(/</g, "&lt;")
250     .replace(/>/g, "&gt;")
251     .replace(/"/g, "&quot;")
252     .replace(/'/g, "&apos;")
```

The name is the only part of the request that comes from you, and it is escaped on the way in, so a name containing `<` or `&` cannot alter the shape of the XML around it. The body asks for one property, a display name, and lets the server default everything else — it is the same body `caldav_make_calendar.py` sends, which is the body Thundermail's Stalwart answered 201 to.

9 Refuse another origin

firefox-addon/popup.js:256–266

```
256 if (permittedOrigin && new URL(url).origin !== permittedOrigin) {
257   throw new Error(
258     `The server answered with an address on ${new URL(url).host}, which is not the ` +
259     "server you asked about. Nothing was sent to it.",
260   );
261 }
262 if (permitted && !(await permitted)) {
263   throw new Error(
264     `Without permission to talk to ${new URL(url).host}, this cannot ask it anything.` ,
265   );
266 }
```

Line 256 compares where this request is going against the origin recorded back at step 2, and refuses if they differ. On *this* path the address was built locally from the home plus a slug, so it can only ever match — the guard exists for the listing path, where an address can come out of a server's reply. It was added after a security review found that a reply naming another host could otherwise have carried your password there.

Line 262 is where the click's permission request is finally waited on. Nothing has been sent yet.

10 The password becomes a header

firefox-addon/popup.js:268–270 and caldav_account.js:239–244

```
268 const headers = { Authorization: basicAuth(user, password) };
269 if (body !== undefined) headers["Content-Type"] = "application/xml; charset=utf-8";
270 if (depth !== undefined) headers.Depth = depth;

239 export function basicAuth(user, password) {
240   const bytes = new TextEncoder().encode(`${user}:${password}`);
241   let binary = "";
242   for (const byte of bytes) binary += String.fromCharCode(byte);
243   return `Basic ${btoa(binary)}`;
244 }
```

HTTP Basic: the username and password joined by a colon and base64-encoded. **Base64 is not encryption** — it is trivially reversible, which is exactly why step 5 refuses plain `http://`. The encoding goes through `TextEncoder` first because `btoa()` throws on anything above U+00FF, and an app password is whatever the provider generated.

The header is built here, used once, and never stored. There is no `Depth` on this request: that header is for listings, and `MKCALENDAR` passes no `depth`.

11 A deadline

firefox-addon/popup.js:272–283

```

276 const deadline = new AbortController();
277 const timer = setTimeout(() => deadline.abort(), SECONDS * 1000);
278 try {
279     return await exchange(method, url, headers, body, deadline.signal);
280 } finally {
281     clearTimeout(timer);
282 }

```

Sixty seconds for the whole exchange, headers and body alike — the same number the Python tools have always used. Without it, a server that accepts the connection and then says nothing would leave the page waiting forever with its buttons disabled.

12 The request

firefox-addon/popup.js:285–304

```

289 response = await fetch(url, {
290     method,
291     headers,
292     body,
293     signal,
294     // No cookies, ever: this signs in with the app password you typed and
295     // nothing else, whatever session the browser happens to be holding.
296     credentials: "omit",
297     cache: "no-store",
298 });
299 } catch (error) {
300     if (error.name === "AbortError") {
301         throw new Error(`${host} did not answer within ${SECONDS} seconds. Nothing was changed.`);
302     }
303     throw new Error(`Could not reach ${host}. Check the address, and that you are online.`);

```

This is the only line in the whole path that touches the network. What goes out, for a calendar called `ticket 7067` on `nemo@thundermail.com`:

```

MKCALENDAR /dav/cal/nemo%40thundermail.com/ticket-7067/ HTTP/1.1
Host: mail.thundermail.com
Authorization: Basic bmVtb0B0aHVuZGVybWFPbC5jb206...
Content-Type: application/xml; charset=utf-8

<?xml version="1.0" encoding="utf-8"?>
<C:mkcalendar xmlns:D="DAV:" xmlns:C="urn:ietf:params:xml:ns:caldav">
  <D:set><D:prop><D:displayname>ticket 7067</D:displayname></D:prop></D:set>
</C:mkcalendar>

```

`credentials: "omit"` means no cookies are attached, so whatever session your browser holds for that site plays no part: the request is authenticated by the app password you typed and by nothing else.

13 The answer

firefox-addon/popup.js:306–318

```

306 if (response.status === 401) {
307     throw new Error(
308         "The server would not accept that username and password. CalDAV here needs an app " +
309         "password; an OAuth2 account password will not do. Some servers also want the bare " +
310         "username rather than the full address.",
311     );
312 }
313 if (response.status === 404) throw new Error("There is nothing at that address – check the
Location field.");
314 if (response.status === 405) throw new Error("Something is already at that address.");
315 if (response.status === 507) throw new Error("The account is out of space.");
316 if (response.status === 403) throw new Error("The server would not allow that.");
317 if (!response.ok && response.status !== 207) throw new Error(`The server answered HTTP
${response.status}.`);
318 return readAtMost(response, MOST);

```

201 Created is the success, and falls through to line 318. The named failures are the ones worth telling apart: 401 is almost always an account password used where an app password is needed; 405 means something is already at that address, which is the server making step 6's check twice; 507 is a full account.

readAtMost() reads the reply but stops at 5 MB, so a server cannot make the page buffer an unbounded amount. For MKCALENDAR the body is usually empty.

14 What you see

firefox-addon/popup.js:215–223

```

215 known.calendars.push({ href: address, path: pathOf(address), name });
216 show();
217 say(
218     `Made "${name}". Subscribe to it in Thunderbird with New Calendar → On the Network, ` +
219     "which lists what the account has.",
220     "good",
221 );
222 document.querySelector("#name").value = "";

```

The new calendar is added to the list on screen without re-asking the server, the name field is cleared so a second calendar is not made by accident, and you are told the one thing the add-on cannot do for you: subscribing in Thunderbird. Making a calendar on the server does not put it in your client.

15 What this path does not do

- **It cannot delete or modify anything.** The add-on contains no DELETE, PUT, PROPPATCH, MOVE, COPY or ACL — only PROPFIND and MKCALENDAR exist in it. Deleting is a command-line tool, on purpose.
- **It cannot overwrite an existing calendar.** Step 6 refuses, and if the listing were somehow stale, the server answers 405 at step 13.
- **It cannot reach any other server.** Step 9 refuses an origin other than the one you granted at step 2.
- **It keeps nothing.** The password lives in a local variable from step 5 to step 12; what happens to it afterwards is covered in [firefox-addon/privacy.pdf](#), with the same kind of line references.

- **It sets no sharing or permissions.** The body at step 8 asks for a display name and nothing else, so the calendar is as private as the account it belongs to.